

MESH SUITE: A UNIFIED BENCHMARK FOR TRIANGLE- AND MESH-BASED NEURAL RENDERING

Kaixuan Zhang¹ Minxian Li^{1*} Mingwu Ren¹ Xiatian Zhu²

¹Nanjing University of Science and Technology ²University of Surrey

ABSTRACT

Triangle- and mesh-based neural rendering aims to bridge neural scene representations and existing graphics engines (*e.g.*, Unity, Blender) by leveraging *triangle* primitives compatible with standard rasterization hardware. There have been several such methods driven by parallel efforts, developed and evaluated under inconsistent settings, with little or no comparison with each other; critically, most even have never used graphics engines for evaluation nor considered deployability in practice – significantly undermining the objective and motivation. To address these issues, we introduce **MeshSuite**, the first benchmark of its kind to enable systematic evaluation of graphics-engine deployment and Non-Engine Render for triangle- and mesh-based neural rendering methods. Importantly, our deployment protocol includes two engine deployment settings: (1) *Standard Deployment* – a conventional opaque mesh pipeline with vertex colors and hardware Z-buffering; (2) *Dedicated Deployment* – adding method-specific engine implementations supporting the retained appearance and compositing features (*e.g.*, alpha blending), so that the characteristics of each specific model can be taken into account. For mesh splatting, we further propose a structural audit of exported surfaces, diagnosing topological and geometric integrity toward downstream graphics assets. We highlight several results: (1) Graphics engine deployment would incur image-quality degradation under both deployment options. Among the triangle- and mesh-based NVS methods studied, MeshSplatting degrades the least under Standard Deployment. (2) The deployment setting matters – Dedicated Deployment can retain the majority of fidelity at a roughly 5–33× slowdown relative to Standard Deployment across the two deployment datasets. (3) About mesh splatting, the current approaches to explicit connectivity and shared indexing are still limited: shared vertex indexing alone does not ensure manifoldness or global connectivity. From this benchmark, we validate that *rasterizability is merely part of graphics readiness*, and highlight the significance of assessing the graphics engine deployment process (*e.g.*, the degree of engine compatibility). Source code is available at <https://github.com/prinasi/MeshSuite>.

1 INTRODUCTION

Reconstructing a scene from multi-view images allows users to explore it from arbitrary viewpoints beyond those originally captured, supporting a wide range of vision and graphics applications, including AR/VR (Broxton et al., 2020), gaming (Lachambre et al., 2017), and robotic perception (Zhu et al., 2022). Recent advances in neural rendering, particularly Neural Radiance Fields (NeRFs) (Mildenhall et al., 2020) and 3D Gaussian Splatting (3DGS) (Kerbl et al., 2023), have enabled increasingly realistic novel-view synthesis (NVS). Specifically, NeRFs use multilayer perceptrons (MLPs) to encode scene density and view-dependent color, synthesizing novel views through volume rendering (Mildenhall et al., 2020). However, their practical use is constrained by high computational costs and the lack of an explicit geometric representation. In contrast, 3DGS represents scenes explicitly with 3D Gaussian primitives and uses efficient differentiable splatting to substantially reduce rendering costs (Kerbl et al., 2023).

*Corresponding author.

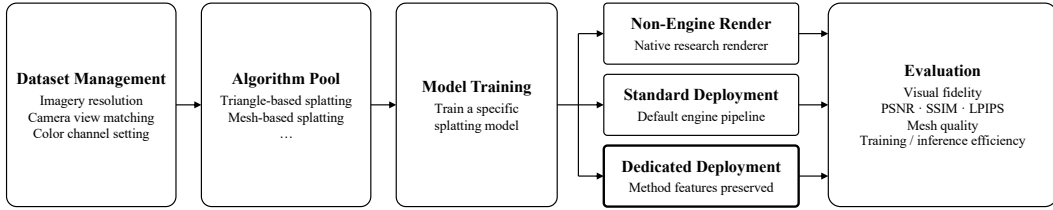


Figure 1: **MeshSuite**: The Dataset Management layer standardizes data formats, including image resolution, camera-view alignment, and color-channel settings. Triangle- and mesh-based splatting methods are integrated into the Algorithm Pool layer and trained through the Model Training layer. The trained model can be evaluated in three ways: Non-Engine Render, Standard Deployment, and Dedicated Deployment; the latter two use a chosen graphics engine. Its performance is then evaluated in the Evaluation layer using a wide range of metrics covering visual fidelity, mesh structure, and training/inference efficiency.

Despite these advances, neither representation is directly compatible with the standard rendering pipelines of established computer graphics (CG) engines, such as Blender (Blender Foundation, 2026) and Unity (Unity Technologies, 2026a). These engines have benefited from decades of development and optimization, providing efficient rendering pipelines and integrated tools for lighting and shadows, material editing, and animation. Their mature ecosystems also offer extensive collections of reusable assets and plugins (Unity Technologies, 2026b; Blender Foundation, 2024), facilitating the integration of reconstructed scenes into practical applications.

To better take advantage of these engines, researchers have increasingly explored triangle- and mesh-based neural rendering methods, which optimize triangle primitives already supported by standard graphics hardware (Held et al., 2026b;a). These methods have demonstrated impressive NVS performance comparable to that of NeRFs and 3DGS, while offering a promising path toward integration with existing CG engines (Held et al., 2026b; Sheng et al., 2025; Held et al., 2026a). However, existing triangle- and mesh-based works follow separate evaluation protocols, with differences in data preprocessing, image formats, evaluation metrics, and metric implementations, and none evaluates the methods in a graphics engine. Using triangles alone also does not ensure engine compatibility. Existing methods use triangles with substantially different features, including continuous or discrete opacity (Held et al., 2026b;a), coverage defined inside or outside a triangle (Sheng et al., 2025; Held et al., 2026b), varying color representations (e.g., view-dependent spherical harmonics (SHs) (Held et al., 2026a) or neural networks (Tojo et al., 2026)), and the presence or absence of neural texture decoders (Tojo et al., 2026; Held et al., 2026b). These variations pose challenges for engine rendering and may require method-specific shaders, buffers, sorting strategies, and runtime state. Mesh splatting evaluations (Held et al., 2026a) also need to assess vertex sharing, manifoldness, and global connectivity.

To address these questions, we introduce **MeshSuite**, a unified benchmark for evaluating triangle- and mesh-based neural rendering (see Fig. 1). This benchmark enables: (a) integration of existing triangle- and mesh-based splatting methods; (b) fair and consistent comparison of different methods in a unified setting; (c) extensive evaluation of both triangle- and mesh-based models during training and inference; (d) Non-Engine Render and two engine deployment settings: *Standard Deployment* - converting a trained model to a standard opaque vertex-colored mesh before rendering in a graphics engine; *Dedicated Deployment* - adding a customized layer to implement method-specific designs, such as specialized appearance representations and alpha compositing.

We note three key findings. *First*, engine deployment reduces visual quality relative to Non-Engine Render, with MeshSplatting (Held et al., 2026a) losing the least quality under Standard Deployment among the studied methods (see Tab. 3). *Second*, Dedicated Deployment produces more visually appealing renderings than Standard Deployment, at the cost of a roughly $5\text{--}33\times$ slowdown across the two deployment datasets (see Tab. 3). *Third*, MeshSplatting’s exported meshes exhibit non-manifold structures and substantial fragmentation (see Tab. 5). These results show that *using triangles alone does not ensure graphics readiness*; appearance preservation, efficiency, and mesh structure must be evaluated together.

Our **contributions** are as follows: **(I)** the first unified benchmark, *MeshSuite*, for evaluating triangle- and mesh-based neural rendering methods; **(II)** Dedicated Deployment of existing methods in a graphics engine, retaining their specific designs during engine rendering; and **(III)** an extensive evaluation of existing models in terms of novel-view synthesis, training/inference cost, and mesh structure.

2 RELATED WORK

Non-mesh-based neural representations. Novel-view synthesis (NVS) aims to generate images of a scene from viewpoints beyond those observed in the input images. A central component of this task is a scene representation that captures both geometry and appearance. Neural radiance fields (NeRFs) represent a scene as a continuous function and synthesize views through volumetric integration (Mildenhall et al., 2020). Later methods improve reconstruction quality and anti-aliasing (Barron et al., 2021; 2022; 2023) or accelerate rendering through explicit, factorized, and hashed structures (Reiser et al., 2024; Yu et al., 2021; Fridovich-Keil et al., 2022; Chen et al., 2022; Müller et al., 2022). 3DGS replaces repeated network queries with optimized anisotropic Gaussians and a dedicated splatting rasterizer (Kerbl et al., 2023). Follow-up work refines Gaussian optimization or encourages surface-aligned geometry (Kheradmand et al., 2024; Huang et al., 2024; Guédon & Lepetit, 2024). These methods establish strong NVS baselines, but their representations are not natively supported by conventional triangle-based graphics pipelines (e.g., Unity and Blender). Integration with existing CG engines therefore requires specialized rendering implementations or conversion to mesh-based assets.

To facilitate integration with conventional graphics pipelines, methods such as MobileNeRF (Chen et al., 2023) and BakedSDF (Yariv et al., 2023) convert learned radiance fields into mesh-based representations. For explicit splats, SuGaR (Guédon & Lepetit, 2024) and GaMeS (Waczyńska et al., 2024) extract mesh surfaces from 3D Gaussians or bind Gaussian primitives to deformable mesh faces. Although these conversion pipelines produce standard mesh assets, they often rely on lossy post-processing (e.g., Marching Cubes, Poisson surface reconstruction, or heuristic simplification) that decouples the optimization target from the final rendering objective.

Triangle- and mesh-based neural representations. Established CG engines, such as Blender and Unity, provide optimized rendering pipelines and integrated tools for scene editing, lighting, and animation (Blender Foundation, 2026; Unity Technologies, 2026a), together with mature ecosystems of reusable assets and plugins (Unity Technologies, 2026b; Blender Foundation, 2024). Triangle meshes are widely supported by these pipelines, making them a natural choice for bringing learned scene representations into existing graphics workflows.

To exploit this support, triangle- and mesh-based neural rendering methods optimize geometry directly represented by triangle primitives (Held et al., 2026b; Sheng et al., 2025; Held et al., 2026a; Tojo et al., 2026). These approaches use either *triangle soups* with independent faces or *indexed meshes* with shared vertices and explicit connectivity. 2DTS (Sheng et al., 2025) and Triangle-Splatting (Held et al., 2026b) optimize independent triangles with soft coverage and alpha blending, DiffSoup (Tojo et al., 2026) uses neural texture features and a color decoder, and MeshSplatting (Held et al., 2026a) introduces shared-vertex connectivity. These works demonstrate the potential of triangle- and mesh-based representations for high-quality NVS and offer promising routes toward graphics-engine integration.

However, using triangle primitives alone does *not* ensure engine compatibility, as method-specific appearance and transparency models may still require additional adaptation. These methods are also developed and reported under different evaluation pipelines, making direct cross-method comparison difficult. Their original evaluations primarily use dedicated research renderers rather than assess the learned representations after deployment in a graphics engine. MeshSuite addresses both gaps through unified evaluation and explicit graphics-engine deployment.

Neural rendering benchmarks. Existing NVS benchmarks (Kulhanek & Sattler, 2025; Liu et al., 2026) standardize evaluation with research renderers but do not assess graphics-engine deployment. MeshSuite extends benchmarking to engine deployment for triangle- and mesh-based NVS methods.

3 MESHsuite FRAMEWORK

MeshSuite provides an end-to-end benchmark spanning evaluation with the original research renderers and deployment in graphics engines. As illustrated in Fig. 1, it evaluates each trained representation under matched camera views, backgrounds, and evaluation metrics across three settings: Non-Engine Render, Standard Deployment, and Dedicated Deployment. Comparisons across these settings reveal how graphics-engine deployment affects visual fidelity and rendering efficiency. For exported indexed meshes, we also assess structural usability, including local/global connectivity and manifoldness. Fig. 6 provides a more detailed overview of MeshSuite’s software architecture.

3.1 EVALUATION PROTOCOL STANDARDIZATION

Fair cross-method comparisons require consistent data and evaluation settings (Kulhanek & Sattler, 2025; Liu et al., 2026). MeshSuite standardizes three aspects:

Camera and viewpoint alignment: For a held-out camera set $\mathcal{C}$, the benchmark standardizes camera coordinate frames, optical axes, field of view, and projection matrices.

Background compositing policy: Prior works adopt inconsistent backgrounds: on DTU (Jensen et al., 2014), Triangle-Splatting uses an unmasked black background, whereas 2DTS uses object masks and a white background. We standardize all methods to the white-background protocol for masked objects. Using the RGBA alpha channel as an opacity matte, both ground-truth target images and novel-view renderings are composited as

$$\mathbf{I}_{\text{white}} = \mathbf{I}_{\text{RGB}} \odot \alpha + (1 - \alpha) \cdot \mathbf{1}. \quad (1)$$

Here, $\mathbf{I}_{\text{RGB}}$ and $\mathbf{I}_{\text{white}}$ denote the input RGB and composited images, respectively; α is the per-pixel opacity matte with values in $[0,1]$; $\odot$ denotes element-wise multiplication, with α applied to all RGB channels; and $\mathbf{1}$ is an all-ones image. The renderer background is also set to white, giving all methods the same evaluation background.

Metric standardization: We use the same implementations of PSNR (Hore & Ziou, 2010), SSIM (Wang et al., 2004), and LPIPS (Zhang et al., 2018) for all methods. LPIPS uses a VGG (Simonyan & Zisserman, 2015) backbone with images normalized to $[-1, 1]$.

3.2 UNIVERSAL TRANSLATION LAYER AND ASSET INTERFACE

Existing triangle- and mesh-based methods encode geometry, appearance, and opacity in method-specific ways. For example, 2DTS (Sheng et al., 2025) and Triangle-Splatting (Held et al., 2026b) use independent triangles with learned opacity, MeshSplatting (Held et al., 2026a) uses shared vertices with view-dependent appearance, and DiffSoup (Tojo et al., 2026) represents appearance through learned features and a neural decoder. Engine deployment therefore requires method-specific interpretation of the trained model’s parameters.

To support deployment through a common interface, MeshSuite introduces a *translation layer* implemented by an adapter $\mathcal{E}_m$ for method m . For each trained model $\mathcal{S}_m$, the adapter maps its parameters to an engine asset $\mathcal{A}_m$ that follows the unified interface:

$$\mathcal{S}_m \xrightarrow{\mathcal{E}_m} \mathcal{A}_m, \quad (2)$$

which covers geometry, appearance, and opacity for different mesh structures. Translation decodes the trained model before deployment, allowing the engine to consume the exported asset independently of the training framework. The asset supplies the geometry and appearance data used by Standard and Dedicated Deployment (see Appendix Fig. 6).

3.3 GRAPHICS ENGINE DEPLOYMENT

As illustrated in Fig. 1, MeshSuite organizes rendering into three evaluation settings: Standard Deployment, Dedicated Deployment, and Non-Engine Render. Non-Engine Render serves as the reference baseline for the two engine-deployment settings.

Method	Standard Deployment		Dedicated Deployment					
	Support	Appearance	Support	View-dep.	Opacity	Soft cov.	Depth sort	Appearance
3DGS		N/A						N/A
2DGS	Fallback	baked color						N/A
2DTS	Fallback	baked SH-DC	Method-aware	✓	✓	✓	✓	SH
Triangle-Splatting	Fallback	baked SH-DC	Method-aware	✓	✓	✓	✓	SH
MeshSplatting	Fallback	baked SH-DC	Method-aware	✓	✓	–	✓	SH
DiffSoup		N/A	Method-aware	✓	–	–	–	MLP

Table 1: Engine support and retained rendering features. *Fallback*: opaque static-color rendering; *Method-aware*: a renderer supporting the listed features. SH-DC denotes the constant SH color component; MLP denotes a multilayer perceptron. ✓/–: feature retained/not used. N/A: unsupported.

Standard Deployment ($\mathbf{I}_m^{\text{std}}$): We render representations as conventional opaque meshes with static vertex colors and hardware Z-buffering. This setting omits method-specific shaders, neural decoders, and alpha blending. DiffSoup is unavailable in this setting.

Dedicated Deployment ($\mathbf{I}_m^{\text{ded}}$): We implement method-specific Unity renderers under the same evaluation protocol that retain each method’s appearance and compositing features (Tab. 1). Comparing them with Non-Engine Render and Standard Deployment reveals how much quality these features preserve and their runtime cost (Tab. 3).

Non-Engine Render ($\mathbf{I}_m^{\text{ne}}$): We render each trained checkpoint $\mathcal{S}_m$ with its official source renderer $\mathcal{R}_m^{\text{ne}}$: $\mathbf{I}_m^{\text{ne}}(c) = \mathcal{R}_m^{\text{ne}}(\mathcal{S}_m, c)$, $\forall c \in \mathcal{C}$, where c denotes a camera in the held-out set $\mathcal{C}$ and $\mathbf{I}_m^{\text{ne}}(c)$ is the corresponding rendered image. This establishes baseline image quality before engine deployment.

Differences from Non-Engine Render. Although Dedicated Deployment retains the same appearance and compositing functions, the engine and source renderers can differ in primitive ordering, coverage, and attribute evaluation. These differences affect transparency and view-dependent appearance, particularly for large or oblique primitives, and contribute to the adaptation gap.

3.4 EVALUATION METRICS

MeshSuite groups evaluation metrics into three categories: NVS quality, mesh quality, and computational cost.

3.4.1 NVS QUALITY

Visual fidelity and deployment gaps: Following Sec. 3.1, we report PSNR (Hore & Ziou, 2010), SSIM (Wang et al., 2004), and LPIPS (Zhang et al., 2018) (VGG backbone, normalized images). For an image-quality metric q (e.g., PSNR, SSIM, or LPIPS), we compute the mean score Q_m^r over the held-out camera set $\mathcal{C}$:

$$Q_m^r = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} q(\mathbf{I}_m^r(c), \mathbf{I}^{\text{gt}}(c)), \quad r \in \{\text{ne}, \text{std}, \text{ded}\}, \quad (3)$$

where m indexes methods, c denotes a held-out camera, and $|\mathcal{C}|$ is the number of evaluated cameras. The images $\mathbf{I}_m^r(c)$ and $\mathbf{I}^{\text{gt}}(c)$ are the rendering of method m under setting r and the corresponding ground truth, respectively. The setting labels ne, std, and ded denote Non-Engine Render, Standard Deployment, and Dedicated Deployment, respectively.

Then we define three pairwise gaps:

$$\Delta Q_m^{\text{adapt}} = |Q_m^{\text{ne}} - Q_m^{\text{ded}}|, \quad \Delta Q_m^{\text{port}} = |Q_m^{\text{ded}} - Q_m^{\text{std}}|, \quad \Delta Q_m^{\text{deploy}} = |Q_m^{\text{ne}} - Q_m^{\text{std}}|. \quad (4)$$

The *adaptation gap* $\Delta Q_m^{\text{adapt}}$ measures the change from Non-Engine Render to Dedicated Deployment; the *portability gap* ΔQ_m^{port} measures the additional change under Standard Deployment; and the *deployment gap* $\Delta Q_m^{\text{deploy}}$ measures the overall change from Non-Engine Render to Standard Deployment. The gaps report absolute differences; the metric values indicate whether quality improves or declines.

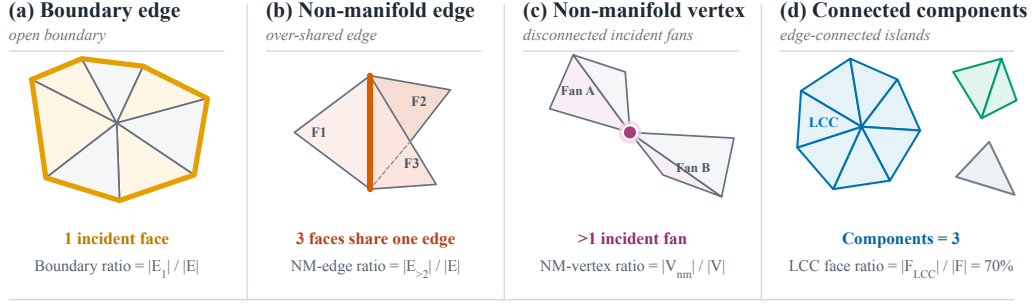


Figure 2: Topology diagnostics for the structural audit. (a) Boundary edges have one incident face. (b) Non-manifold edges have more than two incident faces. (c) Non-manifold vertices have disconnected incident-face fans. (d) Edge-connected components are maximal face sets connected through shared edges.

3.4.2 MESH QUALITY

Surface geometry: We evaluate Chamfer Distance (CD) on DTU, which adopts standard pipelines to isolate the object surface, complementing the white-background image protocol in Sec. 3.1.

Mesh structural integrity: Rendering as triangles does not guarantee a valid geometric surface. Let V , E , and F denote the sets of unique vertices, undirected edges, and triangular faces in an exported mesh, with $|\cdot|$ denoting set cardinality. We audit three complementary aspects:

Local reuse. The ratio $|V|/|F|$ (abbreviated as V/F in tables) measures unique vertices per face, and average valence $2|E|/|V|$ summarizes local adjacency density. With a common export convention, lower $|V|/|F|$ indicates greater vertex sharing.

Local structure. The boundary-edge and non-manifold-edge ratios are $|E_1|/|E|$ and $|E_{>2}|/|E|$, where E_1 and $E_{>2}$ contain edges incident to one and more than two faces, respectively. The non-manifold-vertex ratio is $|V_{nm}|/|V|$, where V_{nm} is the set of vertices whose incident faces fail to form a single valid fan. Valid fans may be closed around an interior vertex or open at a boundary while disconnected fans are non-manifold. (see Fig. 2 for illustration).

Global connectivity. Two faces are considered adjacent if they share an edge. LCC faces and LCC area report the fractions of faces and surface area in the largest connected component (LCC), while Components counts the edge-connected components. We read component counts together with LCC fractions and visual inspection to account for differences in scene size and resolution. These three groups distinguish vertex sharing, local manifoldness, and fragmentation. Appendix Sec. A.3 gives the audit details.

3.4.3 COMPUTATIONAL COST

We audit computational overhead across training and inference. During training, we track peak VRAM (MiB) and optimization time (min). During engine deployment, we report GPU frame latency, wall-clock throughput (FPS), and renderer graphics allocations (MiB) separately. We additionally report serialized asset size (MiB) to characterize storage requirements.

4 BENCHMARK EVALUATION

4.1 EXPERIMENTAL SETUP AND REPRODUCIBILITY

Methods. We evaluate four representative triangle- and mesh-based neural rendering methods, covering different designs in geometry connectivity, opacity, and appearance (Tab. 1).

- **2DTS** (Sheng et al., 2025) replaces Gaussian primitives with independent planar triangle facelets and renders them through differentiable alpha blending. Its opacity function is

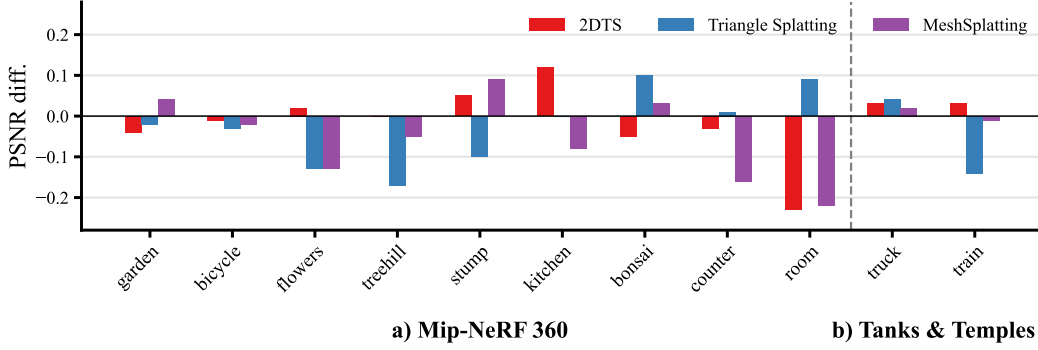


Figure 3: Per-scene PSNR reproduction differences ($\text{PSNR}_{\text{MeshSuite}} - \text{PSNR}_{\text{paper}}$) on (a) Mip-NeRF 360 and (b) Tanks and Temples datasets. The largest relative deviation is below 0.8%.

defined using barycentric coordinates, with a compactness parameter controlling the transition from diffuse splats toward sharper triangle surfaces.

- **Triangle-Splatting** (Held et al., 2026b) directly optimizes independent triangles using a differentiable window function confined to each projected triangle. Learned opacity, view-dependent color, and depth-ordered alpha blending support image-based optimization, while adaptive densification and pruning refine the allocation of triangles.
- **DiffSoup** (Tojo et al., 2026) represents scenes with a compact triangle soup carrying neural texture features and binary opacity. Stochastic opacity masking makes binary opacity trainable within a depth-tested rasterization pipeline, while a neural color decoder maps texture features to rendered colors.
- **MeshSplatting** (Held et al., 2026a) constructs a shared-vertex mesh through restricted Delaunay triangulation and jointly refines geometry and per-vertex spherical-harmonic appearance. Opacity and window parameter schedules progressively guide optimization toward opaque triangles.

The first three methods produce *triangle soups* without explicit shared-vertex connectivity, whereas MeshSplatting produces an *indexed mesh*. We audit the structure of its exported mesh in Sec. 4.4. We additionally include 3DGS (Kerbl et al., 2023) and 2DGS (Huang et al., 2024) as Gaussian-based reference baselines.

Datasets. We evaluate on four complementary datasets spanning real-world environments and object-centric captures. Mip-NeRF 360 (Barron et al., 2022) comprises nine real scenes, with surrounding camera trajectories, cluttered geometry, and large depth variations that challenge reconstruction of both foreground and distant content. For Tanks and Temples (Knapitsch et al., 2017), we use the two outdoor scenes *train* and *truck*, featuring large real-world objects, complex backgrounds, and occlusions. DTU (Jensen et al., 2014) provides laboratory captures of varied objects from 49 or 64 calibrated camera positions per scene, together with structured-light reference scans, enabling geometric evaluation alongside image fidelity. NeRF-Synthetic (Mildenhall et al., 2020) contains eight path-traced object-centric scenes with detailed geometry and realistic non-Lambertian materials. Known camera poses and controlled rendering make it complementary to real captures.

Implementation details. We follow the original training pipelines while applying the shared data and evaluation protocols. Images are downsampled according to each method’s requirements. All training and evaluation use a single NVIDIA GeForce RTX 4090 GPU with 24 GB memory.

Timing and resource measurements. End-to-end FPS includes CPU processing and GPU rendering. We also report GPU median (P50) and 95th-percentile (P95) frame latency, together with renderer graphics allocations.

Reproducibility. Before comparing methods, we check our reproductions against published results and public codebases. Across all 27 Mip-NeRF 360 method–scene pairs with published per-scene results, the mean absolute PSNR difference is only 0.075 dB and the maximum relative deviation

Dataset	Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$	Peak mem. $\downarrow$	Train time $\downarrow$	CD $\downarrow$
Mip-NeRF 360	2DTS	28.16	0.841	0.215	70	10069	46.1	–
	Triangle-Splatting	27.13	0.812	0.227	94	18901	40.5	–
	MeshSplatting	24.72	0.729	0.365	38	23134	41.7	–
	DiffSoup	23.54	0.689	0.322	168	23484	16.8	–
Tanks and Temples	2DTS	23.42	0.853	0.203	101	16061	25.9	–
	Triangle-Splatting	23.08	0.856	0.171	159	11157	24.5	–
	MeshSplatting	20.53	0.756	0.319	66	21321	24.1	–
	DiffSoup	18.90	0.685	0.352	226	23890	15.4	–
NeRF-Synthetic	2DTS	33.65	0.970	0.034	126	3172	16.3	–
	Triangle-Splatting	29.87	0.956	0.055	243	12257	19.1	–
	MeshSplatting	29.50	0.945	0.071	86	23386	13.4	–
	DiffSoup	28.65	0.910	0.076	371	16334	12.2	–
DTU	2DTS	29.66	0.951	0.110	312	2470	21.2	0.63
	Triangle-Splatting	27.78	0.944	0.080	165	4163	13.1	1.27
	MeshSplatting	27.82	0.933	0.106	114	7680	10.7	0.80
	DiffSoup	17.91	0.760	0.263	170	23930	14.5	3.89

Table 2: General *non-engine render* results for NVS quality, surface geometry, and computational cost, averaged over scenes in each dataset.

is below 0.8%. Fig. 3 shows this close agreement in image quality. Appendix Tab. 6 provides dataset-level comparisons against paper values and public-code results.

4.2 STANDARDIZED NON-ENGINE RENDER BENCHMARK

Tab. 2 reports the standardized Non-Engine Render results across all four datasets. On Mip-NeRF 360 (Barron et al., 2022), NeRF-Synthetic (Mildenhall et al., 2020), and DTU (Jensen et al., 2014), 2DTS achieves the highest PSNR (28.16 dB, 33.65 dB, and 29.66 dB), while using the least peak training memory on NeRF-Synthetic and DTU (3172 MiB and 2470 MiB). Triangle-Splatting achieves the lowest LPIPS on Tanks and Temples (0.171) and DTU (0.080). DiffSoup renders and trains fastest on three datasets (up to 371 FPS and 12.2–16.8 min), but has lower image quality (18.90 dB on Tanks and Temples). MeshSplatting uses shared-vertex connectivity and trains fastest on DTU (10.7 min), with lower PSNR on Mip-NeRF 360 (24.72 dB). Thus, *no* representation dominates fidelity, throughput, memory, and training time simultaneously.

Controlled synthetic scenes. On NeRF-Synthetic, 2DTS leads in all three image metrics (33.65 dB PSNR, 0.970 SSIM, and 0.034 LPIPS) and uses the least peak training memory (3,172 MiB). Its 3.78 dB PSNR advantage over Triangle-Splatting comes with lower inference speed: DiffSoup renders at 371 FPS and trains in 12.2 minutes, compared with 126 FPS and 16.3 minutes for 2DTS.

2D novel-view fidelity vs. 3D surface geometry. DTU complements NVS metrics with surface accuracy against reference scans (Tab. 2). Results show that image quality and surface accuracy do not always yield the same ranking. While 2DTS leads in both PSNR (29.66 dB) and CD (0.63 mm), Triangle-Splatting achieves the lowest LPIPS (0.080) with a higher CD of 1.27 mm. MeshSplatting has similar PSNR to Triangle-Splatting (27.82 vs. 27.78 dB) but lower surface error (0.80 mm CD), while DiffSoup has the highest CD (3.89 mm). These results motivate evaluating image quality and surface geometry jointly before graphics deployment.

4.3 RENDERER-CONDITIONED DEPLOYMENT

Tab. 3 compares quality and throughput across the three settings on Mip-NeRF 360 and Tanks and Temples; Appendix Fig. 7 summarizes the PSNR trajectories of the four triangle- and mesh-based methods. Appendix Tab. 11 retains their per-scene Tanks and Temples results. Under our deployment protocol, 3DGS (Kerbl et al., 2023) is unsupported in both engine settings; 2DGS (Huang et al., 2024) lacks Dedicated Deployment because TSDF meshing discards appearance variables used by the source renderer (though its mesh runs under Standard Deployment); and DiffSoup lacks a Standard Deployment opaque path because of its neural feature decoder.

Deployment quality. Standard Deployment incurs substantial visual quality loss by removing method-specific appearance and compositing features. On Mip-NeRF 360, 2DTS and Triangle-

(a) Mip-NeRF 360

Setting	Metric	Reference baselines		Triangle-/mesh-based methods			
		3DGS	2DGS	2DTS	Triangle-Splatting	DiffSoup	MeshSplatting
Non-Engine Render	PSNR $\uparrow$	27.21	26.79	28.16	27.13	23.54	24.72
	SSIM $\uparrow$	0.815	0.796	0.841	0.812	0.689	0.729
	LPIPS $\downarrow$	0.214	0.252	0.215	0.227	0.322	0.365
	FPS $\uparrow$	109	38	70	94	168	38
Standard Deployment	PSNR $\uparrow$	N/A	12.70	12.52	14.77	N/A	19.83
	SSIM $\uparrow$		0.311	0.311	0.246		0.491
	LPIPS $\downarrow$		0.594	0.661	0.656		0.518
	FPS $\uparrow$		2778	3754	2294		943
Dedicated Deployment	PSNR $\uparrow$	N/A	N/A	23.28	22.96	23.15	22.52
	SSIM $\uparrow$			0.699	0.695	0.647	0.589
	LPIPS $\downarrow$			0.313	0.301	0.331	0.457
	FPS $\uparrow$			545	70	1995	157

(b) Tanks and Temples

Setting	Metric	Reference baselines		Triangle-/mesh-based methods			
		3DGS	2DGS	2DTS	Triangle-Splatting	DiffSoup	MeshSplatting
Non-Engine Render	PSNR $\uparrow$	23.74	23.21	23.42	23.08	18.90	20.53
	SSIM $\uparrow$	0.853	0.832	0.853	0.856	0.685	0.756
	LPIPS $\downarrow$	0.169	0.211	0.203	0.171	0.352	0.319
	FPS $\uparrow$	129	92	101	159	226	66
Standard Deployment	PSNR $\uparrow$	N/A	16.10	12.40	11.08	N/A	14.38
	SSIM $\uparrow$		0.525	0.337	0.292		0.506
	LPIPS $\downarrow$		0.515	0.629	0.661		0.490
	FPS $\uparrow$		4263	5480	3928		1528
Dedicated Deployment	PSNR $\uparrow$	N/A	N/A	19.26	19.54	17.02	16.09
	SSIM $\uparrow$			0.646	0.719	0.606	0.587
	LPIPS $\downarrow$			0.296	0.258	0.402	0.435
	FPS $\uparrow$			909	536	2388	300

Table 3: Mean NVS quality and end-to-end rendering throughput across the three evaluation settings on (a) Mip-NeRF 360 and (b) Tanks and Temples. 3DGS and 2DGS are reference baselines. N/A: unsupported.

Splatting lose 15.64 dB and 12.36 dB PSNR relative to Non-Engine Render, while MeshSplatting loses 4.89 dB. Our Dedicated Deployment implementations retain these features and recover 2.69–10.76 dB PSNR over Standard Deployment across these three methods, demonstrating the benefit of our method-specific engine integration. Relative to Non-Engine Render, the remaining PSNR adaptation losses are 4.88 dB and 4.17 dB for 2DTS and Triangle-Splatting, and 0.39 dB and 2.20 dB for DiffSoup and MeshSplatting, respectively.

Standard Deployment removes custom shading and relies on hardware Z-buffering; on Mip-NeRF 360, this causes severe PSNR portability loss (ΔQ_m^{port}) for the soup methods, with drops of 8–11 dB. For example, 2DTS produces large opaque shards on *bonsai*, and Triangle-Splatting exhibits the same failure mode on *treehill* (Fig. 4). The total PSNR drop for 2DTS from Non-Engine Render to Standard Deployment is 15.64 dB on average across Mip-NeRF 360. MeshSplatting has the smallest PSNR portability gap, at 2.69 dB on Mip-NeRF 360. The method ranking therefore changes with the setting: 2DTS has the highest Dedicated Deployment PSNR on Mip-NeRF 360, while Triangle-Splatting leads on Tanks and Temples; MeshSplatting achieves the best Standard Deployment quality among the four triangle- and mesh-based methods on both datasets (Tab. 3 and Appendix Fig. 7). For MeshSplatting, Sec. 4.4 examines whether this rendering quality is accompanied by usable mesh structure.

DiffSoup is unsupported under our Standard Deployment protocol because its appearance is encoded as neural texture features that require a color decoder (marked N/A in Fig. 4). Our Dedicated

Method	Mip-NeRF 360				Tanks and Temples			
	GPU P50 (ms)		Alloc. (MiB)		GPU P50 (ms)		Alloc. (MiB)	
	Std.	Ded.	Std.	Ded.	Std.	Ded.	Std.	Ded.
2DTS	0.3	2.1	54	1140	0.2	1.2	48	832
Triangle-Splatting	0.5	3.8	54	1559	0.3	1.9	48	933
MeshSplatting	1.1	6.4	54	927	0.7	3.4	48	596
DiffSoup	N/A	0.5	N/A	288	N/A	0.4	N/A	276

Table 4: Computational cost summary on both (a) MipNeRF-360 and (b) Tanks and Temples. Std./Ded.: Standard/Dedicated Deployment. P50 values average scene-level medians; Alloc. measures renderer graphics allocations. N/A: unsupported.

Dataset	Local reuse		Local structure			Global connectivity		
	V/F	Val.	Bnd. edges	NM edges ↓	NM verts. ↓	LCC faces ↑	LCC area ↑	Comp. (K)
<i>Large, unbounded scenes</i>								
Mip-NeRF 360	0.523	6.49	0.454	0.178	0.652	0.699	0.689	712.9
Tanks and Temples	0.516	6.49	0.461	0.182	0.665	0.699	0.666	442.8
<i>Object-centric scenes</i>								
DTU	0.428	7.36	0.412	0.222	0.727	0.769	0.775	43.3
NeRF-Synthetic	0.339	8.59	0.372	0.284	0.873	0.804	0.804	65.8

Table 5: Mesh diagnostics for MeshSplatting (Held et al., 2026a), averaged over all scenes in each dataset. Val.: valence; Bnd.: boundary; NM: non-manifold; Comp.: edge-connected components.

Deployment renderer in Unity retains this decoder and achieves a mean PSNR of 23.15 dB on Mip-NeRF 360, compared with 23.54 dB for Non-Engine Render (Tab. 3). The bicycle example in Fig. 4 shows visually similar results under these two settings. Some thin structures, such as bicycle brake cables, remain poorly resolved in both renderings. These shared artifacts are already present in Non-Engine Render and cannot be attributed solely to engine deployment. Supplementary videos compare the rendering settings.

Cross-dataset deployment behavior. On Tanks and Temples, Triangle-Splatting achieves the best Dedicated Deployment quality (19.54 dB PSNR, 0.719 SSIM, and 0.258 LPIPS), whereas DiffSoup achieves the highest Dedicated Deployment throughput (2388 FPS). Among the four triangle- and mesh-based methods, MeshSplatting retains the highest Standard Deployment quality (14.38 dB PSNR, 0.506 SSIM, and 0.490 LPIPS). For the three methods available in both engine settings, Dedicated Deployment improves all three image metrics but reduces throughput by 5.1–7.3×. The same quality–speed trade-off also appears on Mip-NeRF 360.

Quality–efficiency trade-off. Fig. 5 visualizes the empirical Pareto trade-offs across the three evaluation settings on Mip-NeRF 360 and Tanks and Temples. Trajectories trace the progression from Standard Deployment through Dedicated Deployment to the unconstrained Non-Engine baseline. Across the three methods available in both engine settings, Dedicated Deployment substantially recovers visual fidelity lost under naive hardware rasterization, but at a marked throughput cost. For example, on Mip-NeRF 360, Triangle-Splatting recovers 8.19 dB PSNR while rendering 32.8× slower (70 vs. 2294 FPS), and 2DTS recovers 10.76 dB at a 6.9× throughput penalty (545 vs. 3754 FPS). Non-Engine Render provides an additional 4.17–4.88 dB PSNR for 2DTS and Triangle-Splatting, with end-to-end throughput of 70–94 FPS using their source renderers. DiffSoup sits in a distinct regime: lacking an opaque Standard Deployment pipeline, its neural feature decoding maintains consistent PSNR (23.15 vs. 23.54 dB on Mip-NeRF 360) while achieving interactive Dedicated Deployment throughput (1995 FPS). Overall, Dedicated Deployment achieves 70–1995 FPS across all four methods, while Standard Deployment offers ultra-high throughput (943–3754 FPS) at the cost of catastrophic appearance degradation for the soup representations.

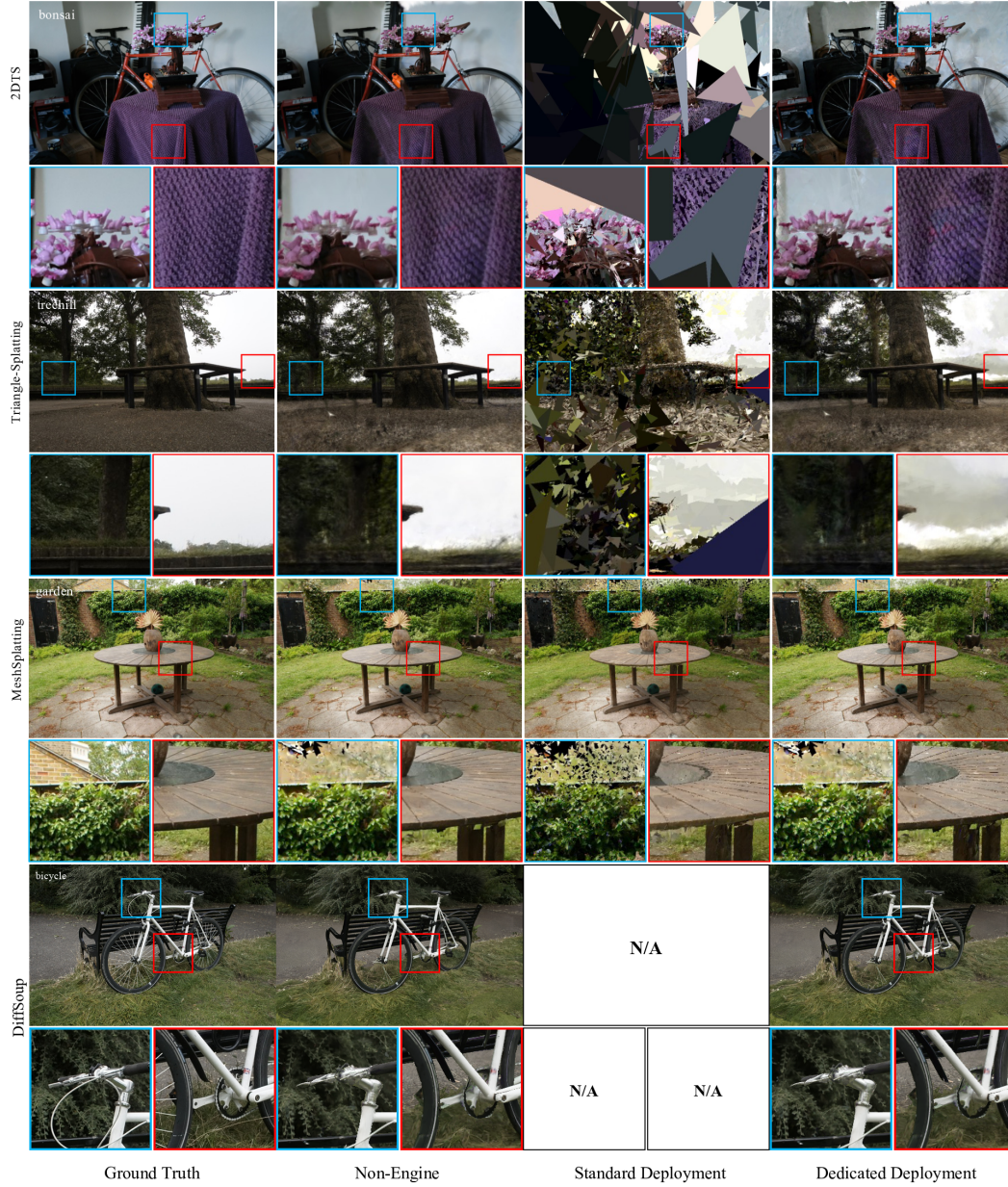


Figure 4: Qualitative NVS comparisons of 2DTS (Sheng et al., 2025), Triangle-Splatting (Held et al., 2026b), MeshSplatting (Held et al., 2026a), and DiffSoup (Tojo et al., 2026) on Mip-NeRF 360. Columns show ground truth, Non-Engine Render, Standard Deployment, and Dedicated Deployment. Colored boxes indicate regions enlarged below each view. DiffSoup is unsupported under the Standard Deployment protocol because its neural texture features require a color decoder.

GPU latency and end-to-end throughput. Tab. 4 summarizes GPU latency and graphics allocations alongside the FPS results in Tab. 3. For Triangle-Splatting under Dedicated Deployment on Mip-NeRF 360, mean GPU P50 latency is 3.8 ms, while 70 FPS corresponds to approximately 14.3 ms per frame. GPU latency measures GPU work; FPS captures the full rendering pipeline, including CPU processing. We therefore use FPS to compare the overall runtime cost of the two deployment settings. Full latency and resource results are given in Appendix Sec. A.4.

Runtime graphics allocations. Dedicated Deployment uses mean renderer graphics allocations of 288–1559 MiB on Mip-NeRF 360 and 276–933 MiB on Tanks and Temples. Standard Deployment

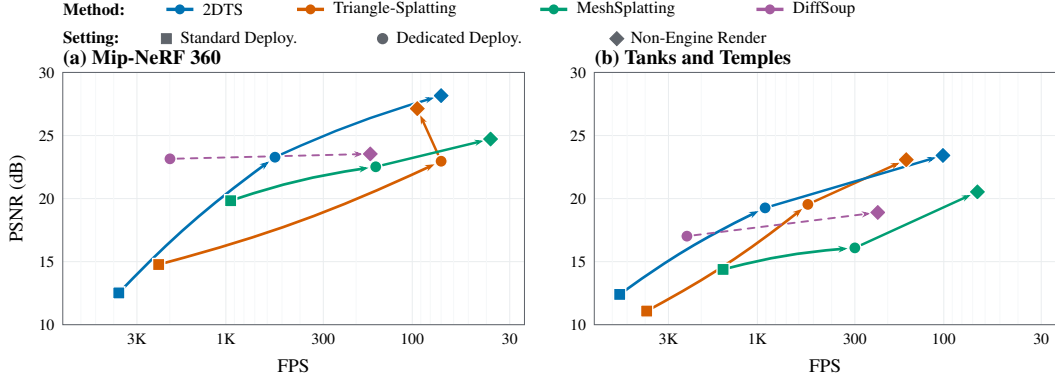


Figure 5: Quality–efficiency trade-off across the three evaluation settings on (a) Mip-NeRF 360 and (b) Tanks and Temples. Trajectories connect Standard Deployment (■) to Dedicated Deployment (●) and Non-Engine Render (◆). DiffSoup lacks an opaque Standard Deployment path and is connected via dashed lines.

reports 54 MiB and 48 MiB, respectively (Tab. 4). These similar Standard Deployment allocations suggest a substantial fixed renderer overhead at the evaluated viewports. The measurements cover renderer graphics resources, rather than total GPU memory. Appendix Sec. A.4 provides per-scene measurements and serialized asset sizes.

4.4 MESH STRUCTURAL RESULTS

We audit MeshSplatting’s exported indexed meshes across all four datasets using the three metric groups defined in Sec. 3.4. Tab. 5 reports the mean of each metric over all scenes in each dataset. We consider the large, unbounded scenes of Mip-NeRF 360 and Tanks and Temples separately from the smaller, object-centric scenes of DTU and NeRF-Synthetic. Appendix Sec. A.3 provides per-scene results and audit details.

Local reuse. Mip-NeRF 360 and Tanks and Temples exhibit similar vertex sharing, with mean $|V|/|F|$ ratios of 0.523 and 0.516 and mean valence of 6.49 on both datasets. The object-centric datasets show greater reuse: DTU and NeRF-Synthetic have lower $|V|/|F|$ ratios of 0.428 and 0.339, with higher valence of 7.36 and 8.59, respectively. Vertex sharing is evident in both scene groups, but its extent differs.

Local structure. Vertex sharing does not ensure manifoldness. On Mip-NeRF 360 and Tanks and Temples, approximately 18% of edges and 65–66% of vertices are non-manifold. On DTU and NeRF-Synthetic, the non-manifold-edge ratios rise to 22% and 28%, and the non-manifold-vertex ratios to 73% and 87%, respectively. Thus, greater reuse on the object-centric datasets coexists with more non-manifold neighborhoods. Boundary edges account for 37–46% of edges across the four datasets, indicating open surfaces while such boundaries can be expected in partially observed scenes. Non-manifold edges and vertices, in contrast, violate local manifold connectivity, and lower ratios indicate better local structural integrity.

Global connectivity. On the large-scene datasets, the largest connected component (LCC) contains approximately 70% of faces, with 69% of surface area on Mip-NeRF 360 and 67% on Tanks and Temples. On DTU and NeRF-Synthetic, the LCC contains 77% and 80% of faces and 78% and 80% of area, respectively. Despite these dominant components, approximately 20–30% of faces and 20–33% of area remain distributed across other components. Together with mean component counts in the tens to hundreds of thousands, these fractions reveal substantial fragmentation in both scene groups. Absolute component counts depend on mesh size and do not provide a structural-quality ranking across datasets. On the two deployment datasets, MeshSplatting’s leading Standard Deployment image quality among the four studied methods (Tab. 3) coexists with these structural defects, showing that visual fidelity does not ensure structural integrity.

5 CONCLUSION

We introduced MeshSuite, a unified benchmark for triangle- and mesh-based neural rendering that connects evaluation with research renderers to graphics-engine deployment. MeshSuite combines standardized evaluation protocols, a unified asset interface, and method-specific Unity renderers to compare Non-Engine Render, Standard Deployment, and Dedicated Deployment. It evaluates visual fidelity and computational cost alongside surface accuracy and mesh structural integrity.

Our experiments show that visual quality and method rankings depend on the deployment setting. Dedicated Deployment better preserves appearance but incurs higher runtime costs than Standard Deployment. Among the four studied methods, MeshSplatting achieves the highest image quality under Standard Deployment on Mip-NeRF 360 and Tanks and Temples, yet its exported meshes exhibit non-manifold structures and substantial fragmentation across both large, unbounded and object-centric scenes. These findings show that triangle rasterization and shared-vertex connectivity alone do not ensure practical graphics usability. Evaluation must jointly consider appearance preservation, rendering efficiency, and the structural requirements of downstream tasks. Future work can extend the benchmark to additional graphics engines and directly assess exported meshes in tasks such as editing, remeshing, and collision handling.

REFERENCES

- Jonathan T. Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P. Srinivasan. Mip-NeRF: A multiscale representation for anti-aliasing neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 5855–5864, 2021.
- Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Mip-NeRF 360: Unbounded anti-aliased neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5470–5479, 2022.
- Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Zip-nerf: Anti-aliased grid-based neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 19697–19705, 2023.
- Blender Foundation. Extensions platform beta release. Online, 2024. URL <https://code.blender.org/2024/05/extensions-platform-beta-release/>. Accessed: 2026-10-05.
- Blender Foundation. Blender. Online, 2026. URL <https://www.blender.org/>.
- Michael Broxton, John Flynn, Ryan Overbeck, Daniel Erickson, Peter Hedman, Matthew DuVall, Jason Dourgarian, Jay Busch, Matt Whalen, and Paul Debevec. Immersive light field video with a layered mesh representation. *ACM Transactions on Graphics*, 39(4), 2020.
- Anpei Chen, Zexiang Xu, Andreas Geiger, Jingyi Yu, and Hao Su. TensorRF: Tensorial radiance fields. In *Computer Vision – ECCV 2022*, pp. 333–350. Springer, 2022.
- Zhiqin Chen, Thomas Funkhouser, Peter Hedman, and Andrea Tagliasacchi. Mobilenerf: Exploiting the polygon rasterization pipeline for efficient neural field rendering on mobile architectures. In *The Conference on Computer Vision and Pattern Recognition*, 2023.
- Sara Fridovich-Keil, Alex Yu, Matthew Tancik, Qinlong Chen, Benjamin Recht, and Angjoo Kanazawa. Plenoxels: Radiance fields without neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5501–5510, 2022.
- Antoine Guédon and Vincent Lepetit. SuGaR: Surface-aligned gaussian splatting for efficient 3D mesh reconstruction and high-quality mesh rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5354–5365, 2024.
- Jan Held, Sanghyun Son, Renaud Vandeghen, Daniel Rebain, Matheus Gadelha, Yi Zhou, Anthony Cioppa, Ming C. Lin, Marc Van Droogenbroeck, and Andrea Tagliasacchi. MeshSplatting: Differentiable rendering with opaque meshes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 7320–7329, 2026a.
- Jan Held, Renaud Vandeghen, Adrien Deliege, Abdullah Hamdi, Daniel Rebain, Silvio Giancola, Anthony Cioppa, Andrea Vedaldi, Bernard Ghanem, Andrea Tagliasacchi, and Marc Van Droogenbroeck. Triangle splatting for real-time radiance field rendering. In *International Conference on 3D Vision*, 2026b.
- Alain Hore and Djemel Ziou. Image quality metrics: Psnr vs. ssim. In *2010 20th international conference on pattern recognition*, pp. 2366–2369. IEEE, 2010.
- Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2D gaussian splatting for geometrically accurate radiance fields. In *ACM SIGGRAPH 2024 Conference Papers*, pp. 1–11. ACM, 2024.
- Rasmus Jensen, Anders Dahl, George Vogiatzis, Engin Tola, and Henrik Aanæs. Large scale multi-view stereopsis evaluation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 406–413, 2014.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4):1–14, 2023.
- Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Yang-Che Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo. In *Advances in Neural Information Processing Systems*, 2024. Spotlight Presentation.
- Arno Knapitsch, Jaesik Park, Qian-Yi Zhou, and Vladlen Koltun. Tanks and temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics*, 36(4), 2017.

- Jonas Kulhanek and Torsten Sattler. Nerfbaselines: Consistent and reproducible evaluation of novel view synthesis methods. *Advances in Neural Information Processing Systems*, 38:127378–127403, 2025.
- Sébastien Lachambre, Sébastien Lagarde, and Cyril Jover. Photogrammetry workflow. Technical report, Unity Technologies, 2017. URL https://unity3d.com/files/solutions/photogrammetry/Unity-Photogrammetry-Workflow_2017-07_v2.pdf.
- Xiang Liu, Yimin Zhou, Jinxiang Wang, Yujun Huang, Shuzhao Xie, Shiyu Qin, Mingyao Hong, Jiawei Li, Yaowei Wang, Zhi Wang, Shu-Tao Xia, and Bin Chen. Splatwizard: A benchmark toolkit for 3d gaussian splatting compression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Findings*, pp. 2261–2271, 2026.
- Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *Proceedings of the European Conference on Computer Vision*, pp. 405–421, 2020.
- Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics*, 41(4):1–15, 2022.
- Christian Reiser, Songyou Peng, Yiyi Liao, and Andreas Geiger. KiloNeRF: Speeding up neural radiance fields with thousands of tiny MLPs. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 14335–14345, 2024.
- Kaifeng Sheng, Zheng Zhou, Yingliang Peng, and Qianwei Wang. 2D triangle splatting for direct differentiable mesh training, 2025.
- Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015.
- Kenji Tojo, Bernd Bickel, and Nobuyuki Umetani. Diffsoup: Direct differentiable rasterization of triangle soup for extreme radiance field simplification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 8353–8363, 2026.
- Unity Technologies. Unity. Online, 2026a. URL <https://unity.com/>.
- Unity Technologies. Introduction to the asset store. Online documentation, 2026b. URL <https://docs.unity.com/en-us/asset-store/introduction>.
- Joanna Waczyńska, Piotr Borycki, Sławomir Tadeja, Jacek Tabor, and Przemysław Spurek. Games: Mesh-based adapting and modification of gaussian splatting. *arXiv preprint arXiv:2402.01459*, 2024.
- Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4): 600–612, 2004.
- Lior Yariv, Peter Hedman, Christian Reiser, Dor Verbin, Pratul P Srinivasan, Richard Szeliski, Jonathan T Barron, and Ben Mildenhall. Baked sdf: Meshing neural sdfs for real-time view synthesis. In *ACM SIGGRAPH 2023 conference proceedings*, pp. 1–9, 2023.
- Alex Yu, Ruilong Li, Matthew Tancik, Hao Li, Ren Ng, and Angjoo Kanazawa. PlenOctrees for real-time rendering of neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 5752–5761, 2021.
- Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 586–595, 2018.
- Zihan Zhu, Songyou Peng, Viktor Larsson, Weiwei Xu, Hujun Bao, Zhaopeng Cui, Martin R. Oswald, and Marc Pollefeys. NICE-SLAM: Neural implicit scalable encoding for SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12786–12796, 2022.

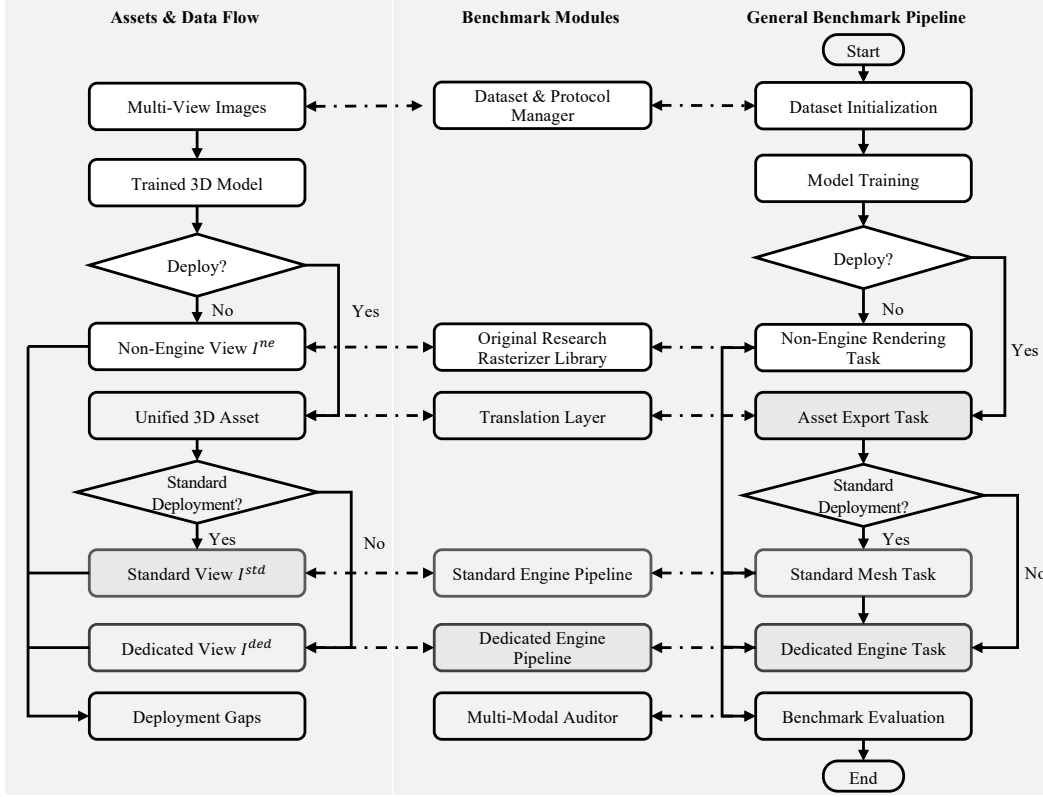


Figure 6: **Software architecture and implementation overview of MeshSuite.** The benchmark organizes data, method integration, evaluation, deployment, and structural auditing into modular components that support the three evaluation settings.

A APPENDIX

A.1 SOFTWARE ARCHITECTURE AND PIPELINE IMPLEMENTATION

Fig. 6 details the modular software architecture of *MeshSuite*, expanding the conceptual overview in Fig. 1. The infrastructure is built with decoupling and modularity as first principles, ensuring that model training, format translation, engine-native deployment, and multi-dimensional evaluation operate independently through clearly specified interfaces.

Modular Component Design. MeshSuite is organized into five cohesive subsystems: (1) *Dataset Management*: Standardizes canonical camera frames, optical axes, field-of-view definitions, and resolution downsampling across datasets. For masked object captures (e.g., DTU), this layer automatically computes object alpha mattes and applies the white-background compositing convention specified in Eq. 1. (2) *Algorithm Pool and Training*: Provides standardized training harnesses wrapping the official implementations of 2DTS (Sheng et al., 2025), Triangle-Splatting (Held et al., 2026b), MeshSplatting (Held et al., 2026a), and DiffSoup (Tojo et al., 2026). It follows each method’s original training pipeline while applying shared data and evaluation protocols under a common single-GPU hardware environment. (3) *Universal Translation Layer*: The central translation bridge implementing method-specific adapters $\mathcal{E}_m$. It parses trained model parameters and extracts explicit geometry, opacity fields, and appearance representations into standardized intermediate assets $\mathcal{A}_m$. (4) *Graphics Engine Deployment*: Native engine renderers implemented in Unity using C# Scriptable Render Pipelines and custom HLSL shaders. This layer provides both Standard Deployment and Dedicated Deployment. (5) *Evaluation and Audit Suite*: Automated evaluation scripts computing standard 2D image fidelity metrics (e.g., PSNR, SSIM, LPIPS), hardware timing profiles (GPU P50/P95 latency percentiles and memory allocations), and geometric mesh topology audits.

Asset Serialization and Decoupled Execution. Under the Universal Translation Layer, trained models are converted into portable asset files prior to engine execution. Independent triangle soup models (2DTS, Triangle-Splatting, DiffSoup) are serialized into flat vertex arrays and attribute textures, whereas indexed meshes (MeshSplatting) are serialized with shared vertex coordinate tables, triangle face indices, and per-vertex SH coefficients. Crucially, the resulting asset bundles are completely self-contained and ingested directly by the graphics engine, decoupling real-time deployment from PyTorch, CUDA extension compilers, and Python runtime overhead. This architectural separation allows new neural rendering representations to be benchmarked by implementing a single translation adapter, and enables future deployment to other graphics engines (*e.g.*, Unreal Engine, Blender) without modifying upstream training pipelines.

A.2 REPRODUCIBILITY COMPARISONS AND CROSS-IMPLEMENTATION ANALYSIS

Tab. 6 complements the per-scene PSNR reproduction differences illustrated in Fig. 3 with comprehensive dataset-level evaluations comparing our reproduced metrics against published paper numbers and official public repositories. Sec. 4.1 outlines the primary experimental setup.

Quantitative Reproduction Fidelity. For 2DTS, Triangle-Splatting, and MeshSplatting, MeshSuite closely matches published dataset-level PSNR on Mip-NeRF 360 and Tanks and Temples. On Mip-NeRF 360, the mean absolute PSNR difference across their 27 scene-method pairs is only 0.075 dB, and the maximum relative deviation across these pairs is below 0.8%¹. Specifically, 2DTS reproduces at 28.16 dB PSNR (vs. 28.18 dB in the paper and 28.51 dB in the public repository), Triangle-Splatting achieves 27.13 dB (vs. 27.17 dB reported and 27.20 dB in public code), and MeshSplatting achieves 24.72 dB (vs. 24.78 dB in both paper and code). For DiffSoup, where per-scene Mip-NeRF 360 results were not reported in the original paper, MeshSuite is closer to the public codebase in dataset-level PSNR (23.54 dB vs. 23.69 dB) than to the paper (24.76 dB); on Tanks and Temples, it achieves 18.90 dB vs. 18.38 dB in public code. LPIPS differences are larger for several methods, so close PSNR agreement does not imply equal agreement across all image metrics.

Sources of Discrepancies and Impact of Standardization. Differences between published figures, public repositories, and MeshSuite may reflect preprocessing, camera conventions, background compositing, metric implementations, and numerical variation. We do not isolate their individual contributions: (1) *Image Downsampling Kernels*: Image interpolation filters can affect reconstructed image values; MeshSuite applies shared data preparation while retaining the downsampling requirements of each method (Sec. 4.1). (2) *Camera Intrinsic and Principal Points*: Differences in principal point handling and coordinate conventions can affect alignment; MeshSuite enforces standardized projection matrices aligned with graphics conventions. (3) *Background Compositing Conventions*: On masked datasets such as DTU, prior works alternated between evaluating against unmasked black backgrounds and masked white backgrounds; MeshSuite standardizes all evaluations to the alpha matte white-background protocol (Eq. 1). (4) *Metric Implementations and Numerical Variation*: MeshSuite uses a common metric implementation, including VGG-based LPIPS (Sec. 3.1), while parallel floating-point accumulation can also introduce numerical variation. The comparisons document agreement and remaining differences rather than establishing a single cause for them.

A.3 STRUCTURAL AUDIT DETAILS AND PER-SCENE RESULTS

Sec. 3.4 defines the structural metrics. For connected-component analysis, we exclude faces with repeated vertex indices from the face-adjacency graph and apply no component-size threshold. Tabs. 7 and 8 report per-scene results for all four datasets. Each mean row is the arithmetic average of scene-level metrics. The main-text summary is in Tab. 5. We highlight the following key findings:

Large, unbounded scenes. Mip-NeRF 360 and Tanks and Temples exhibit comparable vertex reuse ($|\mathbf{V}|/|\mathbf{F}| \approx 0.52$) and non-manifold ratios (18% NM edges, 65–66% NM vertices). Within Mip-NeRF 360, *kitchen* achieves the highest LCC face and area fractions (77% and 78%), whereas *flowers* records the lowest (65% and 63%), showing substantial variation in primary component

¹DiffSoup is omitted since it does not report per-scene metrics in its original paper.

Dataset	Method	Paper			Public			MeshSuite		
		PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
Mip-NeRF 360	2DTS	28.18	0.842	0.218	28.51	0.851	0.215	28.16	0.841	0.215
	Triangle-Splatting	27.17	0.814	0.192	27.20	0.813	0.190	27.13	0.812	0.227
	MeshSplatting	24.78	0.728	0.310	24.78	0.728	0.310	24.72	0.729	0.365
	DiffSoup	24.76	0.748	0.204	23.69	0.691	0.241	23.54	0.689	0.322
Tanks and Temples	2DTS	23.39	0.853	0.204	23.44	0.853	0.171	23.42	0.853	0.203
	Triangle-Splatting	23.14	0.857	0.143	23.09	0.856	0.144	23.08	0.856	0.171
	MeshSplatting	20.52	0.745	0.287	20.58	0.758	0.272	20.53	0.756	0.319
	DiffSoup	—	—	—	18.38	0.671	0.299	18.90	0.685	0.352

Table 6: Reproduction comparisons on Mip-NeRF 360 (Barron et al., 2022) and Tanks and Temples (Knapitsch et al., 2017). Bold indicates the best result within each dataset and implementation. DiffSoup does not report Tanks and Temples results in its paper.

(a) Mip-NeRF 360

Scene	Local reuse		Local structure			Global connectivity		
	V/F	Val.	Bnd. edges	NM edges $\downarrow$	NM verts. $\downarrow$	LCC faces $\uparrow$	LCC area $\uparrow$	Comp.
bicycle	0.573	5.90	0.453	0.165	0.574	0.689	0.668	720,695
flowers	0.624	5.62	0.485	0.144	0.553	0.651	0.633	758,308
garden	0.475	6.85	0.427	0.196	0.700	0.732	0.730	771,271
stump	0.571	5.92	0.453	0.165	0.569	0.682	0.643	762,169
treehill	0.594	5.77	0.463	0.157	0.565	0.676	0.658	752,539
room	0.465	7.22	0.461	0.183	0.752	0.709	0.711	706,507
counter	0.464	7.08	0.456	0.197	0.731	0.703	0.705	616,675
kitchen	0.408	7.56	0.404	0.238	0.746	0.771	0.780	514,944
bonsai	0.536	6.46	0.481	0.157	0.680	0.675	0.676	813,180
Mean	0.523	6.49	0.454	0.178	0.652	0.699	0.689	712,921

(b) Tanks and Temples

Scene	Local reuse		Local structure			Global connectivity		
	V/F	Val.	Bnd. edges	NM edges $\downarrow$	NM verts. $\downarrow$	LCC faces $\uparrow$	LCC area $\uparrow$	Comp.
truck	0.537	6.26	0.465	0.178	0.629	0.688	0.667	392,140
train	0.495	6.72	0.458	0.187	0.700	0.711	0.665	493,467
Mean	0.516	6.49	0.461	0.182	0.665	0.699	0.666	442,804

Table 7: Per-scene mesh diagnostics for MeshSplatting (Held et al., 2026a) on large, unbounded scenes: (a) Mip-NeRF 360 and (b) Tanks and Temples. Val.: valence; Bnd.: boundary; NM: non-manifold; Comp.: edge-connected components. Ratios are fractions; component counts are un-scaled.

consolidation. On Tanks and Temples, *truck* and *train* allocate approximately 67% of area to the LCC despite divergent NM-vertex ratios (63% vs. 70%).

(a) DTU								
Scene	Local reuse		Local structure			Global connectivity		
	V/F	Val.	Bnd. edges	NM edges ↓	NM verts. ↓	LCC faces ↑	LCC area ↑	Comp.
scan24	0.427	7.35	0.421	0.225	0.725	0.751	0.757	68,693
scan37	0.365	8.13	0.418	0.278	0.799	0.792	0.799	75,446
scan40	0.506	6.47	0.420	0.180	0.645	0.747	0.754	57,598
scan55	0.478	6.72	0.405	0.190	0.664	0.755	0.759	42,646
scan63	0.383	7.94	0.395	0.248	0.786	0.795	0.802	32,738
scan65	0.435	7.34	0.444	0.216	0.738	0.752	0.763	47,295
scan69	0.436	7.21	0.419	0.220	0.719	0.770	0.776	46,788
scan83	0.394	7.81	0.396	0.233	0.772	0.793	0.802	23,631
scan97	0.429	7.34	0.434	0.222	0.727	0.765	0.774	41,168
scan105	0.449	7.02	0.392	0.202	0.695	0.764	0.771	36,507
scan106	0.425	7.50	0.428	0.207	0.735	0.746	0.751	49,957
scan110	0.388	7.76	0.399	0.255	0.761	0.782	0.789	27,437
scan114	0.379	7.90	0.399	0.259	0.760	0.797	0.801	38,662
scan118	0.486	6.61	0.397	0.186	0.658	0.743	0.747	30,660
scan122	0.434	7.28	0.408	0.211	0.721	0.775	0.782	30,328
Mean	0.428	7.36	0.412	0.222	0.727	0.769	0.775	43,304

(b) NeRF-Synthetic								
Scene	Local reuse		Local structure			Global connectivity		
	V/F	Val.	Bnd. edges	NM edges ↓	NM verts. ↓	LCC faces ↑	LCC area ↑	Comp.
chair	0.330	8.86	0.367	0.278	0.891	0.806	0.795	54,755
drums	0.335	8.82	0.387	0.271	0.886	0.779	0.782	70,768
figus	0.398	7.32	0.367	0.282	0.798	0.801	0.815	20,357
hotdog	0.303	8.94	0.316	0.333	0.882	0.843	0.846	44,522
lego	0.312	9.21	0.367	0.293	0.916	0.801	0.806	117,899
materials	0.367	8.34	0.423	0.248	0.861	0.782	0.787	96,682
mic	0.340	8.67	0.392	0.274	0.866	0.800	0.790	33,617
ship	0.323	8.58	0.356	0.295	0.883	0.817	0.814	87,746
Mean	0.339	8.59	0.372	0.284	0.873	0.804	0.804	65,793

Table 8: Per-scene mesh diagnostics for MeshSplatting (Held et al., 2026a) on object-centric datasets: (a) DTU and (b) NeRF-Synthetic. Val.: valence; Bnd.: boundary; NM: non-manifold; Comp.: edge-connected components. Ratios are fractions; component counts are unscaled.

DTU Results and Local Defects. Across the fifteen DTU scans, LCC face and area fractions range from 74.3–79.7% and 74.7–80.2%, respectively (mean 76.9% and 77.5%), while non-manifold-edge ratios average 22.2% (18.0–27.8%). For instance, *scan37* concentrates 79.2% of faces and 79.9% of area into the LCC, yet 27.8% of edges and 79.9% of vertices violate manifold conditions. Concentrated component mass therefore does not preclude local topological defects in the optimized mesh.

NeRF-Synthetic Results and High Vertex Reuse. On NeRF-Synthetic, LCC face and area fractions both average 80.4%, while non-manifold-edge ratios average 28.4% (peaking at 33.3% in *hotdog*), and NM-vertex ratios average 87.3% (91.6% in *lego*). Although vertex reuse is elevated ($|\mathbf{V}|/|\mathbf{F}| = 0.339$, mean valence 8.59), it coexists with high non-manifold ratios rather than ensuring regular 2-manifold surfaces.

Structural Audit Analysis and Practical Implications. Across all four benchmarks, our geometric audit highlights three critical structural challenges: (1) *Non-Manifold Structure*: The audited MeshSplatting exports exhibit pervasive non-manifold edges and vertices, with dataset mean ratios of 18–28% and 65–87%, respectively. (2) *Surface Fragmentation*: While the LCC encompasses approximately 67–80% of area, the remaining surface is distributed across many smaller components; mean total component counts range from 43.3k on DTU to 712.9k on Mip-NeRF 360. (3) *Downstream Usability*: These defects motivate additional validation for workflows such as collision handling,

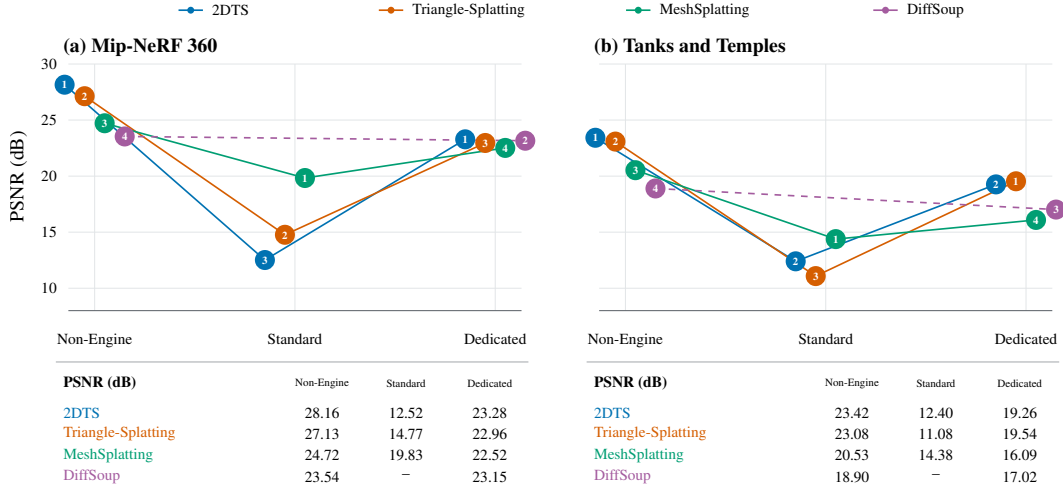


Figure 7: Deployment gaps for the four triangle- and mesh-based methods on (a) Mip-NeRF 360 and (b) Tanks and Temples. Each line follows one method across Non-Engine Render, Standard Deployment, and Dedicated Deployment.

UV unwrapping, and simplification. The structural audit does not directly measure performance in these tasks but it reveals that *novel-view fidelity does not ensure structural usability*.

A.4 DEPLOYMENT RESULTS AND RESOURCE MEASUREMENTS

The following sections, figures, and tables provide the complete deployment-gap visualization, dataset-level runtime summaries, and scene-level measurements supporting Sec. 4.3. The primary deployment comparison and four-dataset Non-Engine Render benchmark are reported in main-text Tabs. 3 and 2; reproduction comparisons are documented in Appendix Sec. A.2.

A.4.1 DEPLOYMENT GAPS AND CROSS-SETTING TRAJECTORIES

Fig. 7 visualizes the PSNR trajectories of the four studied triangle- and mesh-based NVS representations across Non-Engine Render, Standard Deployment, and Dedicated Deployment on Mip-NeRF 360 and Tanks and Temples. Tab. 9 summarizes the corresponding GPU latency percentiles (P50 and P95) and active graphics memory allocations, providing an extensive dataset-level view that expands the compact summary in main-text Tab. 4.

Cross-Setting Trajectory Analysis. Across both deployment benchmarks, the visual quality trajectories reveal a consistent two-tier pattern: (1) *Severe Deployment Degradation for Triangle Soups*: Unconstrained triangle soup methods (2DTS and Triangle-Splatting) rely heavily on soft blending kernels and continuous opacity to represent fine structures and semi-transparent boundaries. When forced into Standard Deployment—where triangles are treated as fully opaque surfaces with hardware Z-buffering—these methods suffer dataset-mean PSNR drops of 11.02–15.64 dB relative to Non-Engine Render. In scenes with complex foliage or fine geometric thinness (e.g., *treehill* or *bonsai*), overlapping transparent triangles collapse into severe opaque, jagged polygon shards (see Fig. 4). (2) *Surface-Constrained Resilience in MeshSplatting*: In contrast, MeshSplatting exhibits far greater resilience under Standard Deployment, dropping by only 4.89 dB on Mip-NeRF 360 (19.83 dB vs. 24.72 dB) and 6.15 dB on Tanks and Temples (14.38 dB vs. 20.53 dB). Because its geometry is constrained to an indexed mesh surface, disabling alpha compositing causes significantly fewer occluding artifacts, allowing it to lead the four triangle- and mesh-based methods under Standard Deployment despite lower initial Non-Engine Render fidelity.

Runtime Resource Trade-Offs. As reported in Tab. 9, Dedicated Deployment preserves image quality at the expense of substantial computational and memory overhead. On Mip-NeRF 360, Dedicated Deployment increases GPU P50 latency by approximately 5.8–7.6 $\times$ compared to Standard Deployment, based on the displayed dataset means (2.1–6.4 ms vs. 0.3–1.1 ms), while mean ren-

Dataset	Method	Deployment	GPU P50 (ms) ↓	GPU P95 (ms) ↓	Graphics allocation (MiB) ↓
Mip-NeRF 360	2DTS	Dedicated	2.1	2.2	1140
	Triangle-Splatting		3.8	4.0	1559
	MeshSplatting		6.4	6.6	927
	DiffSoup		0.5	0.6	288
	2DTS	Standard	0.3	0.3	54
	Triangle-Splatting		0.5	0.5	54
	MeshSplatting		1.1	1.1	54
Tanks and Temples	2DTS	Dedicated	1.2	1.3	832
	Triangle-Splatting		1.9	1.9	933
	MeshSplatting		3.4	3.5	596
	DiffSoup		0.4	0.4	276
	2DTS	Standard	0.2	0.2	48
	Triangle-Splatting		0.3	0.3	48
	MeshSplatting		0.7	0.7	48

Table 9: Mean GPU latency and renderer graphics allocations by dataset. P50/P95 are means of scene-level percentiles; graphics allocations are distinct from serialized asset size and total GPU memory.

renderer graphics allocations for these three methods increase from 54 MiB to 927–1559 MiB. DiffSoup represents a distinct operating point: utilizing neural texture decoding rather than millions of independent Gaussian-like primitives, it achieves sub-millisecond GPU latency (0.4–0.5 ms) and mean renderer graphics allocations of 276–288 MiB, though at the cost of lower overall image fidelity (17.02–23.15 dB).

A.4.2 MIP-NeRF 360 PER-SCENE RUNTIME PROFILING

Tab. 10 provides detailed per-scene measurements of GPU frame time percentiles (median P50 and 95th-percentile P95) and active renderer graphics allocations across all nine Mip-NeRF 360 scenes under both Dedicated and Standard Deployment settings. Sec. 4.3 discusses the primary deployment trade-offs.

Indoor vs. Outdoor Memory Scaling. The per-scene breakdown highlights a stark contrast in resource demands between scene categories: (1) *Dedicated Deployment Allocations*: For indoor scenes (*bonsai*, *counter*, *kitchen*, *room*), graphics allocations remain moderately bounded, averaging 571 MiB for 2DTS, 1023 MiB for Triangle-Splatting, and 855 MiB for MeshSplatting. In contrast, outdoor scenes (*bicycle*, *flowers*, *garden*, *stump*, *treehill*) require dramatic scaling, averaging 1595 MiB for 2DTS (peaking at 1602 MiB on *garden*) and 1988 MiB for Triangle-Splatting (peaking at 2136 MiB on *flowers*). This scaling is driven by unbounded background geometry and dense vegetative foliage requiring millions of alpha-blended primitives. DiffSoup, by contrast, maintains an almost invariant footprint of 280–298 MiB across both indoor and outdoor environments. (2) *Standard Deployment Overhead*: Under Standard Deployment, renderer graphics allocations collapse to an almost identical 51–57 MiB (mean 54 MiB) across all nine scenes and all three supported methods. This uniformity suggests a substantial fixed overhead in the measured renderer graphics resources at the evaluated viewport resolution; it does not establish the total engine memory footprint.

Latency Stability and Tail Profiling. GPU latency percentiles show limited spread on the evaluated scenes. Under Dedicated Deployment, DiffSoup’s displayed P95 exceeds P50 by at most 0.1 ms on every scene, while 2DTS has differences of 0.2 ms on *counter* and *kitchen* and at most 0.1 ms elsewhere. Triangle-Splatting and MeshSplatting have maximum differences of 0.5 ms and 0.3 ms, respectively (e.g., Triangle-Splatting on *counter*: 3.2 ms P50 vs. 3.7 ms P95). These measurements describe GPU latency variability; they do not establish deterministic end-to-end frame pacing.

Representation Features and Runtime Differences. The methods differ in geometry and appearance evaluation, but the reported timings do not isolate costs by rendering stage: (1) *Geometry*

Dedicated Deployment												
Scene	2DTS			Triangle-Splatting			MeshSplatting			DiffSoup		
	P50	P95	Alloc.	P50	P95	Alloc.	P50	P95	Alloc.	P50	P95	Alloc.
<i>bonsai</i>	1.4	1.4	572	3.1	3.2	1159	6.9	7.2	975	0.4	0.4	298
<i>counter</i>	1.7	1.9	571	3.2	3.7	1043	5.5	5.7	779	0.4	0.4	298
<i>kitchen</i>	1.7	1.9	571	2.5	2.8	1001	5.8	5.9	779	0.4	0.5	298
<i>room</i>	1.4	1.4	572	2.2	2.3	890	6.7	6.9	887	0.3	0.3	298
Indoor mean	1.6	1.7	571	2.8	3.0	1023	6.2	6.4	855	0.4	0.4	298
<i>bicycle</i>	2.6	2.6	1599	4.9	5.0	2084	6.3	6.5	956	0.7	0.7	280
<i>flowers</i>	2.6	2.7	1594	5.0	5.1	2136	6.0	6.2	935	0.6	0.7	280
<i>garden</i>	2.6	2.6	1602	4.5	4.5	1930	7.9	8.2	1078	0.7	0.8	281
<i>stump</i>	2.6	2.6	1579	4.3	4.4	1838	6.4	6.7	982	0.6	0.7	280
<i>treehill</i>	2.7	2.7	1601	4.6	4.7	1950	6.3	6.5	971	0.7	0.7	280
Outdoor mean	2.6	2.6	1595	4.7	4.8	1988	6.6	6.8	984	0.7	0.7	280
All-scene mean	2.1	2.2	1140	3.8	4.0	1559	6.4	6.6	927	0.5	0.6	288

Standard Deployment												
Scene	2DTS			Triangle-Splatting			MeshSplatting			DiffSoup		
	P50	P95	Alloc.	P50	P95	Alloc.	P50	P95	Alloc.	P50	P95	Alloc.
<i>bonsai</i>	0.2	0.2	57	0.4	0.4	57	1.1	1.1	57		N/A	
<i>counter</i>	0.2	0.2	57	0.4	0.4	57	1.0	1.0	57		N/A	
<i>kitchen</i>	0.2	0.3	57	0.4	0.4	57	1.1	1.1	57		N/A	
<i>room</i>	0.2	0.2	57	0.3	0.3	57	1.1	1.1	57		N/A	
Indoor mean	0.2	0.2	57	0.4	0.4	57	1.1	1.1	57		N/A	
<i>bicycle</i>	0.3	0.3	51	0.5	0.6	51	1.0	1.0	51		N/A	
<i>flowers</i>	0.3	0.3	52	0.6	0.6	52	0.9	0.9	52		N/A	
<i>garden</i>	0.3	0.4	52	0.5	0.5	52	1.3	1.3	52		N/A	
<i>stump</i>	0.3	0.3	52	0.5	0.5	52	1.1	1.1	52		N/A	
<i>treehill</i>	0.3	0.3	52	0.5	0.5	52	1.0	1.0	52		N/A	
Outdoor mean	0.3	0.3	52	0.5	0.5	52	1.1	1.1	52		N/A	
All-scene mean	0.3	0.3	54	0.5	0.5	54	1.1	1.1	54		N/A	

Table 10: Per-scene GPU latency and renderer graphics allocations on Mip-NeRF 360. P50 and P95 are in ms; Alloc. is in MiB. Indoor, outdoor, and all-scene means average scene-level measurements. DiffSoup has no supported Standard Deployment path.

Organization: 2DTS and Triangle-Splatting use independent triangles, whereas MeshSplatting uses shared vertex indices. These designs imply different geometry-processing requirements; the aggregate GPU latency measurements do not establish which stage dominates. (2) *Appearance and Compositing:* MeshSplatting combines SH evaluation with alpha compositing. DiffSoup decodes appearance from neural texture features without per-primitive alpha compositing or depth sorting (Tab. 1) and achieves sub-millisecond mean GPU latency. The measurements do not isolate the contribution of each feature to this runtime difference.

A.4.3 TANKS AND TEMPLES PER-SCENE DEPLOYMENT RESULTS

Tab. 11 reports per-scene image fidelity, throughput, GPU latency, and resource metrics for *train* and *truck*.

Per-Scene Fidelity and Robustness. Triangle-Splatting achieves the highest Dedicated Deployment quality on *truck* (20.63 dB PSNR, 0.733 SSIM, 0.215 LPIPS), whereas 2DTS leads in PSNR on *train* (18.79 dB), while Triangle-Splatting has the highest SSIM (0.705) and lowest LPIPS (0.301). Under Standard Deployment, Triangle-Splatting degrades severely on *train* (dropping 9.18 dB from Dedicated Deployment to 9.26 dB), while MeshSplatting retains 13.88 dB on *train* and 14.88 dB on *truck*, leading the four triangle- and mesh-based methods (14.38 dB mean). DiffSoup provides extreme throughput exceeding 2300 FPS (2345–2431 FPS) at 0.4 ms GPU latency, albeit with lower visual fidelity (17.02 dB mean).

Setting	Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$	P50 $\downarrow$	P95 $\downarrow$	Render $\downarrow$	Asset $\downarrow$
<i>train</i>									
Standard	2DTS	11.68	0.313	0.655	6643	0.2	0.2	48	234
	Triangle-Splatting	9.26	0.257	0.710	3587	0.3	0.3	48	556
	MeshSplatting	13.88	0.485	0.519	1313	0.8	0.8	48	434
	DiffSoup				N/A				
Dedicated	2DTS	18.79	0.650	0.330	1222	0.8	0.8	562	234
	Triangle-Splatting	18.44	0.705	0.301	469	2.1	2.2	1025	556
	MeshSplatting	15.67	0.572	0.465	249	4.0	4.1	673	434
	DiffSoup	15.38	0.546	0.482	2431	0.4	0.4	276	258
<i>truck</i>									
Standard	2DTS	13.12	0.361	0.603	4316	0.2	0.2	48	469
	Triangle-Splatting	12.89	0.327	0.613	4269	0.2	0.2	48	440
	MeshSplatting	14.88	0.528	0.462	1743	0.6	0.6	48	346
	DiffSoup				N/A				
Dedicated	2DTS	19.72	0.642	0.262	596	1.7	1.7	1103	469
	Triangle-Splatting	20.63	0.733	0.215	604	1.7	1.7	841	440
	MeshSplatting	16.52	0.602	0.405	350	2.9	2.9	518	346
	DiffSoup	18.66	0.666	0.321	2345	0.4	0.4	276	258
<i>Mean</i>									
Standard	2DTS	12.40	0.337	0.629	5480	0.2	0.2	48	352
	Triangle-Splatting	11.08	0.292	0.661	3928	0.3	0.3	48	498
	MeshSplatting	14.38	0.506	0.490	1528	0.7	0.7	48	390
	DiffSoup				N/A				
Dedicated	2DTS	19.26	0.646	0.296	909	1.2	1.3	832	352
	Triangle-Splatting	19.54	0.719	0.258	536	1.9	1.9	933	498
	MeshSplatting	16.09	0.587	0.435	300	3.4	3.5	596	390
	DiffSoup	17.02	0.606	0.402	2388	0.4	0.4	276	258

Table 11: Per-scene quality, throughput, and resources on Tanks and Temples. P50/P95 are GPU latency in ms; Render and Asset are renderer graphics allocation and serialized asset size in MiB. Mean rows average the two scenes, including scene-level latency percentiles. N/A: unsupported under the benchmark protocol.

Serialized Asset Footprint vs. Active Graphics Allocation. Comparing active renderer memory against serialized asset size reveals key trade-offs: (1) Under Standard Deployment, measured renderer graphics allocations are 48 MiB while serialized assets range from 234–556 MiB. These allocations do not measure total GPU memory. (2) Dedicated Deployment reports renderer graphics allocations of 276 MiB for DiffSoup and 518–1103 MiB for the other three methods (*e.g.*, 1103 MiB for a 469 MiB asset in 2DTS on *truck*). Practical edge deployment must account for renderer allocations, total GPU memory, and storage constraints separately.

Engine Deployment Recommendations. (1) *Standard Pipelines*: MeshSplatting offers higher Standard Deployment image quality than the two splatting soup methods on the evaluated datasets, with mean renderer graphics allocations of 48–54 MiB. (2) *High-Framerate XR*: DiffSoup reaches 2388 FPS on Tanks and Temples and 1995 FPS on Mip-NeRF 360 without primitive sorting overhead; these benchmark measurements do not directly establish XR performance. (3) *Topological Regularization*: Incorporating geometric priors during training is vital to penalize non-manifold edges and fragmentation for downstream physics.